Lesson 7 RRCLite PWM Servo Control Principle and Source Code Analysis

1. PWM Servo Description

A servo is a common motor drive device used to control the position, speed, and acceleration in a mechanical system. It typically consists of a motor, a control circuit, and a feedback system.

In terms of type, servos can be categorized into analog servos, PWM (pulse width modulation) servos, and bus servos. This section mainly focuses on PWM servos.

PWM servos use pulse width modulation signals to control the angular position of the servo.

In simple terms, a PWM servo controls the angular position by varying the pulse width of the signal. The range of the pulse width determines the extreme angular positions of the servo, while the period of the control signal determines the refresh frequency of the signal. By adjusting the duration of the pulse width, precise control of the servo position can be achieved.

PWM achieves a level state between 0 and 1, meaning that when PWM is output, the voltage can be any value between 0 and 3.3V/5V.

When using PWM control, it is essential to know two parameters of the object being controlled:

Its PWM frequency (corresponding to the timer cycle)

Its operating pulse width (corresponding to the PWM duty cycle)

The control signal enters the signal modulation chip from the receiver channel to obtain a DC bias voltage. Inside, a reference circuit generates a reference signal with a period of 20ms and a width of 1.5ms. The obtained DC bias voltage is compared with the voltage of the potentiometer to produce a voltage difference output. Finally, the positive and negative outputs of the voltage difference are sent to the motor driver chip to determine the motor's forward

and reverse rotation. When the motor speed is constant, the potentiometer is driven to rotate through the cascade reduction gear, ensuring the voltage difference becomes zero, and the motor stops rotating.

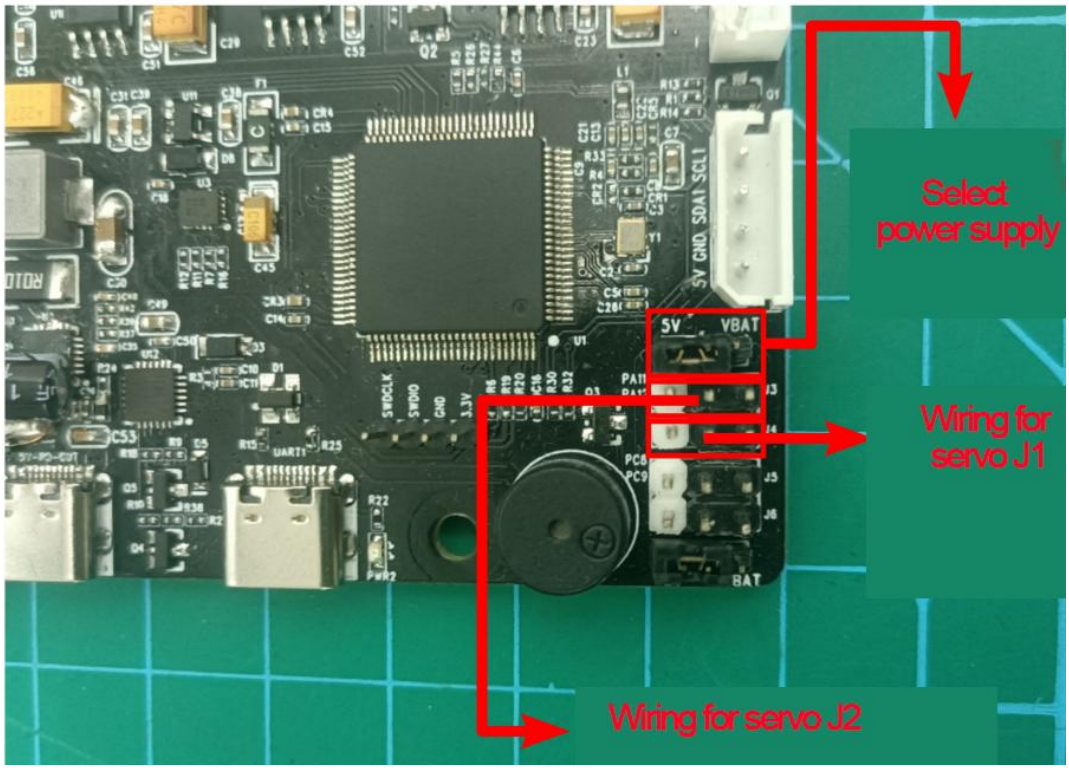
2. Wiring Method:

2.1 Required Stuffs:

1. stm32 controller
2. LD-1501MG digital servo

2.2 Connection Instructions

1. The diagram below shows the reserved interfaces J1 and J2 provided by the STM32 main control board for the PWM servo. Use a jumper cap to connect the top row of pins in the figure to different power sources, shorting them to the 5V end (inserting them into the left two pins). The two servo pins below are powered by 5V voltage.



2. Connect the servo's 3-pin cable to the corresponding J2 or J1 interface, as

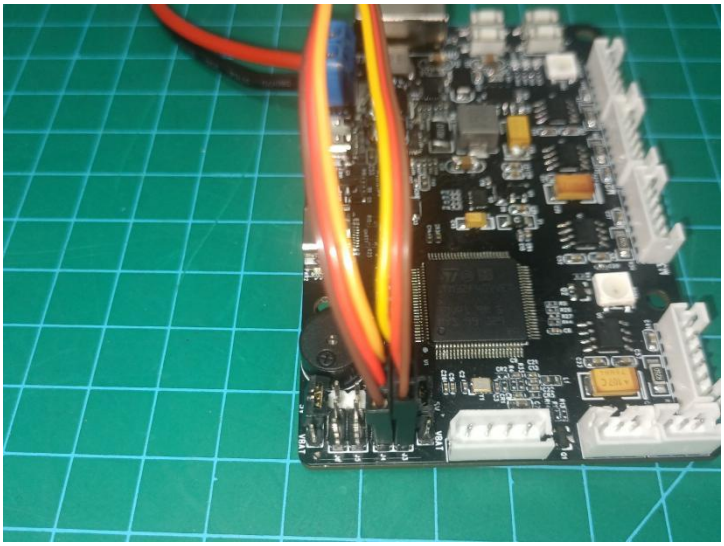
shown below:

The connection method is as follows:

White -> Signal line (SUBS)

Red -> Power supply (+5V)

Black -> Ground (−)



Connect to J1 and J2

The STM32 main control board can be powered by our 7.4V lithium battery pack. For detailed connection instructions, please refer to "Lithium Battery Usage and Precautions" in the current document path.

Attention: The servo motor must be connected according to the positions shown in the document diagram. Avoid connecting it incorrectly or mistakenly to prevent burning out the motor. Our company will not take any responsibility!!!

---

3. Program Download

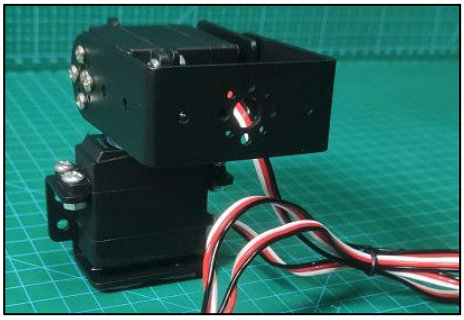
The sample program can be found in “Lesson 7 RRCLite PWM Servo Control Principle and Source Code Analysis/RosRobotControllerLite_ros”.

For the steps and precautions required for programming burning, please refer to “2. STM32 Development Fundamentals/Lesson 3 Project Compilation and Download” document. Simply compile the “RosRobotControllerLite_ros”

program located in the current document path to generate the hex file and proceed with the burning process.

4. Program Outcome

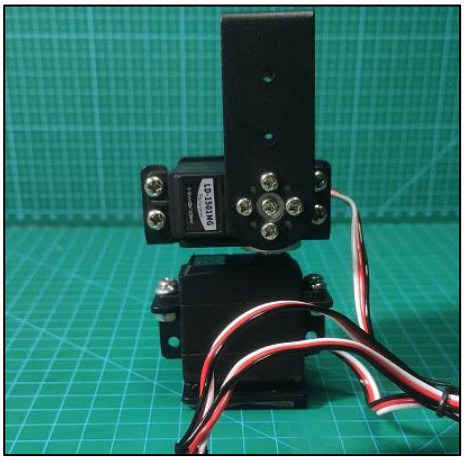
Based on the set PWM servo pulse width values, the servo rotates to the corresponding angles, as shown in the following diagram:



Pulse width:

Servo J1(down): 0

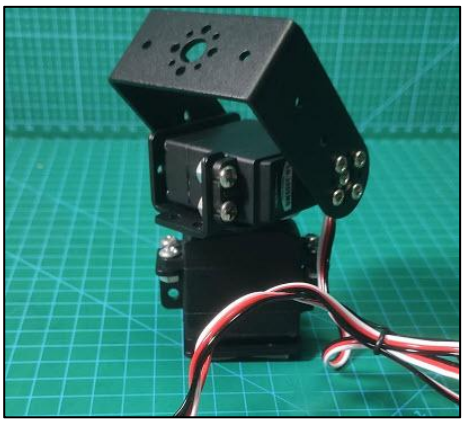
ServoJ2 (up): 0



Pulse width:

Servo J1(down): 1500

ServoJ2 (up): 1500



Pulse width:

Servo J1(down): 2000

ServoJ2 (up): 2000

Pulse width:

Servo J1(down): 2500

ServoJ2 (up): 2500

5. Code Analysis

5.1 Header File

File location: Hiwonder/Peripherals

Object PWMServoObjectTypeDef:

```
17
18  typedef struct PWMServoObject  PWMServoObjectTypeDef;
19
```

Call the PWM control object generated by the PWMServoObject structure.

PWMServoObject Structure:

```c
/**
 * @brief PWM servo object structure
 */
struct PWMServoObject {
    int id;     /**< @brief servo ID */
    int offset; /**< @brief servo deviation */
    int target_duty;  /**< @brief target pulse width */
    int current_duty; /**< @brief current pulse width */
    int duty_raw;     /**< @brief machine pulse width, which is the final pulse width written to the timer and includes the offset.

    /* variables needed for speed control */
    uint32_t duration;    /**< @brief The time that is used for the servo to move from the current angle to the specified angle. It
    float duty_inc; /**< @brief the increment in pulse width for each position update */
    int  inc_times; /**< @brief the number of times the pulse width needs to be incremented */
    bool is_running;  /**< @brief it indicates whether the servo is in the process of motion */
    bool duty_changed; /**< @brief it indicates whether the pulse width has been changed */

    /* provide an external hardware abstracction interface */
    void (*write_pin)(uint32_t new_state); /* set IO port voltage level */
};
```

This structure is used to configure PWM control objects.

Parameters:

Id: Servo ID

offset: Servo offset

target_duty: Target pulse width

current_duty: Current pulse width

duty_raw: Raw pulse width, the width finally written into the timer

Duration: Time for the servo to move from the current angle to the specified angle, controlling the speed

duty_inc: Pulse width increment for each position update

inc_times: Number of increments needed

is_running: Whether the servo is in the process of rotation

duty_changed: Whether the pulse width has been changed

Function write_pin:

This function is used for setting the IO port level. After initialization, this function will be used to set the 0-3 PWM servos.

It has the parameter 'new_state' representing the current pin's high or low level.

Declare function pwm_servo_object_init:

```
/**
 * @brief servo object initialization
 * @param object Pointer to the servo object to be initialized
 * @retval None.
*/
void pwm_servo_object_init(PWMServoObjectTypeDef *object);
```

This function initializes the PWM servo object. Instantiation requires initializing the servo object pointer.

Declare function pwm_servo_duty_compare:

```
/**
 * @brief Servo pulse width control
 * @detials Calculates the pulse width change and the pulse width required for the current speed, and implements the control.
 * @param self Pointer to the servo object to be controlled
 * @retval None.
*/
void pwm_servo_duty_compare(PWMServoObjectTypeDef *self);
```

This function controls the PWM servo pulse width, calculates the pulse width change and the pulse width required for the current speed, and implements control. It needs to be called every 50ms and requires setting the servo object pointer for control.

Declare function pwm_servo_set_position:

```
/**
 * @brief Set servo angle
 * @detials Used to set the angle of the servo. Actually, it only sets a few internal variables of the servo object and does
 * @param self Pointer to the servo object to be controlled
 * @param New pulse width of the servo, an integer value between 500 and 2500
 * @param None.
*/
void pwm_servo_set_position (PWMServoObjectTypeDef *self, uint32_t duty, uint32_t duration);
```

Set the angle of the PWM servo. Actually, it only sets several internal variables of the servo object and does not immediately control the servo. The actual servo control is calculated and controlled by pwm_servo_duty_compare.

Declare function pwm_servo_set_offset：

```
/**
 * @brief Set servo deviation
 * @param self Pointer to the servo object to be controlled
 * @param New servo deviation
 * @param None.
*/
void pwm_servo_set_offset(PWMServoObjectTypeDef *self, int offset);
```

Set the offset of the PWM servo, requiring the servo object pointer for control.

5.2 Source Code

File location: Hiwonder/Peripherals

Function pwm_servo_duty_compare:

```
void pwm_servo_duty_compare(PWMServoObjectTypeDef *self)   //Pulse width change comparison and speed control
{
    // Recalculate the servo control parameters based on the newly set target
    if(self->duty_changed) {
        self->duty_changed = false;
        self->inc_times = self->duration / 20; // Calculate the number of increments needed
        if(self->target_duty > self->current_duty) { /* Calculate the total position change */
            self->duty_inc = (float)(-(self->target_duty - self->current_duty));
        } else {
            self->duty_inc = (float)(self->current_duty - self->target_duty);
        }
        self->duty_inc /= (float)self->inc_times; /* Calculate the position increment per control cycle */
        self->is_running = true;  // The servo starts to move
    }

    // control the servo to rotate to the newly set position
    if(self->is_running) {
        --self->inc_times;
        if(self->inc_times == 0) {
            self->current_duty = self->target_duty;   // assign the set value to the current value to ensure the final position is correct
            self->is_running = false; // The servo stops running when it reaches the set position
        } else {
            self->current_duty = self->target_duty + (int)(self->duty_inc * self->inc_times);
        }
    }
    self->duty_raw = self->current_duty + self->offset; // the action needs to add the servo deviation
}
```

This function is used for controlling pulse width change comparison and speed control.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled.

Firstly, it calculates the control parameters of the servo according to the newly set target. Then, it controls the servo to gradually rotate to the corresponding new position based on the newly set position. Finally, when the last increment occurs, the new position will be assigned to the current position, ensuring the final position is correct.

It is worth noting that when the target pulse width target_duty is greater than self->current_duty, the calculated duty_inc = (-(self->target_duty - self->current_duty)) is a negative value. Afterwards, it is self-divided by self->inc_times.

When entering the if(self->is_running) code block for the first time, self->current_duty is equal to self->target_duty + (int)(self->duty_inc * self->inc_times) = self->current_duty. As the number of times entering the code block increases and self->inc_times decreases, the value

of self->current_duty increases, approaching target_duty, and there is no logical error in the code.

Function: pwm_servo_set_position:

```
void pwm_servo_set_position (PWMServoObjectTypeDef *self, uint32_t duty, uint32_t duration)
{
    duration = duration < 20 ? 20 : (duration > 30000 ? 30000 : duration); /* limit the shortest/longest movement time */
  duty = duty > 2500 ? 2500 : (duty < 500 ? 500 : duty); /* limit the maximum/minimum pulse width value */
    self->target_duty = duty;
    self->duration = duration;
    self->duty_changed = true; /* Mark the target position as changed and let "pwm_servo_duty_compare" calculate new motion parameters */
}
```

This function is used to set the rotation position of the servo.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled, rotation position (pulse width), required time.

Firstly, it is necessary to limit the duration of the movement as well as the maximum and minimum values of the pulse width. Then, assign the corresponding rotation position and time, as well as the switch to control the servo rotation, to the parameters corresponding to the instantiated PWMServoObjectTypeDef object.

Function pwm_servo_set_offset:

```
void pwm_servo_set_offset(PWMServoObjectTypeDef *self, int offset)
{
    offset = offset < -100 ? -100 : (offset > 100 ? 100 : offset); /* Limit the minimum/maximum deviation.
    self->offset = offset;
}
```

This function is used to set the offset of the servo.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled, PWM servo offset.

Here, the magnitude limit of the offset will be set. When the servo rotates, this parameter will be used for adjustment.

Function pwm_servo_object_init:

```
void pwm_servo_object_init(PWMServoObjectTypeDef *obj)
{
    memset(obj, 0, sizeof(PWMServoObjectTypeDef));
    obj->current_duty = 1500; /* default position */
    obj->duty_raw = 1500;     /* actual pulse width */
}
```

This function is the initialization function for the PWM servo PWMServoObjectTypeDef instantiated object.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled.

Firstly, new memory is allocated for initialization, and then the initial position of the PWM servo is set.

5.3 Initialization

File location: Hiwonder/Porting

Instantiation of PWM servo objects:

```
6  PWMServoObjectTypeDef *pwm_servos[4];
```

Instantiate objects of the PWMServoObjectTypeDef structure for PWM servos 1-4.

Setting servo:

```
 8  static void pwm_servo1_write_pin(uint32_t new_state)
 9  {
10      HAL_GPIO_WritePin(PWM_SERVO_1_GPIO_Port, PWM_SERVO_1_Pin, (GPIO_PinState)new_state);
11  }
12
13  static void pwm_servo2_write_pin(uint32_t new_state)
14  {
15      HAL_GPIO_WritePin(PWM_SERVO_2_GPIO_Port, PWM_SERVO_2_Pin, (GPIO_PinState)new_state);
16  }
17
18  static void pwm_servo3_write_pin(uint32_t new_state)
19  {
20      HAL_GPIO_WritePin(PWM_SERVO_3_GPIO_Port, PWM_SERVO_3_Pin, (GPIO_PinState)new_state);
21  }
22
23  static void pwm_servo4_write_pin(uint32_t new_state)
24  {
25      HAL_GPIO_WritePin(PWM_SERVO_4_GPIO_Port, PWM_SERVO_4_Pin, (GPIO_PinState)new_state);
26  }
27
```

Here, four functions are used to configure the interfaces of four PWM servos. They call HAL_GPIO_WritePin to set the port number, pin, and high or low level for each PWM servo.

Parameter: 'new_state' represents the current pin's high or low level.

PWM servo initialization:

```
void pwm_servos_init(void)
{
    for(int i = 0; i < 4; ++i) {
      pwm_servos[i] = LWMEM_CCM_MALLOC(sizeof(PWMServoObjectTypeDef));
        pwm_servo_object_init(pwm_servos[i]);  // Initialize PWM servo object memory
    }
    pwm_servos[0]->write_pin = pwm_servo1_write_pin;
    pwm_servos[1]->write_pin = pwm_servo2_write_pin;
    pwm_servos[2]->write_pin = pwm_servo3_write_pin;
    pwm_servos[3]->write_pin = pwm_servo4_write_pin;

    __HAL_TIM_SET_COUNTER(&htim13, 0);
    __HAL_TIM_CLEAR_FLAG(&htim13, TIM_FLAG_UPDATE);
    __HAL_TIM_CLEAR_FLAG(&htim13, TIM_FLAG_CC1);
    __HAL_TIM_ENABLE_IT(&htim13, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE_IT(&htim13, TIM_IT_CC1);
    __HAL_TIM_ENABLE(&htim13);
}
```

This function is used for initializing PWM servos.

Firstly, it initializes the memory of existing PWM objects and resets them to their initial positions.

Then, it assigns the function for setting servo levels to the write_pin function of the PWM servo object.

Finally, it sets up the timer.